

Architecture

Layered Architecture

Layered Architecture separates components into layers, where each layer only accesses the one below.

It allows incremental development and makes outer layers easier to change as only inner layers interact with hardware or the operating system.

Example:

Internet banking systems – this is the best use case for a Layered Architecture type. It has a Presentation layer – the User Interface (UI) that the user sees, the business logic layer, how the UI works and why, the application/service layer and then the data access layer.

Most appropriate uses:

- Best used when the system has clearly separated responsibilities.
- Being layered, it ensures high maintenance and readability.
- Allows multiple teams to work on different logics simultaneously.
- Beneficial if business logic changes rapidly, faster than infrastructure changes—ideal for financial institutions.
- Layered architecture is stable and centralised.

Why this pattern is the best for the example:

- Security rules and business logic must be isolated from the UI
- Databases should not be accessed directly by users
- Changes to the interface do not affect core business rules
- Each layer can be tested independently
- Customers interact only with the presentation layer
- Transaction processing occurs in the business layer
- Customer records are stored in the data layer

Repository Architecture

Repository Architecture is when all data is kept in a central repository and all components are separated but can interact through the repository. A great use for this kind of architecture is for lots of data. There is a bi-directional flow of data, to and from the repository which mean input and output of data is easy. However, a drawback is that can mean a problem occurring in the repository, could impact the whole system.

Example:

Git (a version control system) is a great example for Repository Architecture. Git acts as a central repository, where all changes are stored. Developers can work in parallel to push and pull changes to a stored repository which ensures consistency and history tracking is kept a key feature.

Most appropriate uses:

- Systems with shared data – Multiple components need access to the same data store.
- Collaboration-heavy projects – Teams work independently but must integrate changes.
- Data persistence and history required – Systems needing versioning or audit trails.

Why this pattern is the best for the example:

- Centralised Data Management – Single source of truth for all code.
- Version Control – Tracks changes and allows rollback easily.
- Supports Collaboration – Multiple developers work individually and merge safely.
- Audit – Every change is recorded with author and timestamp.
- Data Consistency – Ensures code integrity across teams.

Client-server Architecture

Client-server Architecture is used when a set of services is developed to run on servers. The clients access the servers, over a network to make use of the system. The network allows multiple clients to access the servers at one time. It is heavily reliable on network connectivity, if it does not have this, the user is unable to access the servers, and thus it's systems services.

Example:

Web applications, such as Gmail is a great example where a Client-server architecture is best used. This allows the UI and user input whilst keeping the server, which manages the data, business logic and processing to keep each separated. It has a central location for all data storage and processing. It can connected to via Web, App and Desktop).

Most Appropriate Uses:

- Multiple users need centralised access to services or data
- Resources must be controlled from a central location
- Systems require remote access over a network
- Security and data consistency are important

Why this pattern is the best for the example:

- Users access the system from clients (web browsers or mobile apps)
- Google servers store emails and process requests centrally
- The server handles authentication, storage, and synchronization

Pipe and filter Architecture

Pipe and filter architecture is where the data provided goes through a series of transformations as it flows from one component to the other. The pattern is useful for data processing. Transformations can occur in a series as well as in parallel.

Example:

A good example for a pipe and filter architecture is an ETL (Extract, Transform and Load) pipeline. This is where the data flows through stages: extraction -> cleaning -> transformation -> loading. It forms the backbone of business intelligence and analytics.

Most Appropriate Uses:

- Data needs to go through multiple independent processing steps.
- Each stage can operate independently of others.
- Reusability of processing components is important.
- The system must support streaming or batch processing.
- New processing stages may need to be added easily without changing existing filters.

Why this pattern is the best for the example:

- Each stage (extract, transform, load) can be developed and tested independently.
- Data flows in a predictable sequence through reusable components.
- New transformations can be added as filters without modifying other stages.

Two examples of systems combining two or more patterns, with strengths and limitations of these combines systems.

Example 1:

Amazon.com (E-commerce platform)

- Client-Server:
 - The browser or mobile app is the client.
 - Amazon's web servers handle requests, send pages, and process transactions.
- Layered Architecture:
 - Presentation Layer: Web pages and mobile UI.
 - Business Logic Layer: Handles orders, recommendations, payment processing.
 - Data Access Layer: Communicates with the databases storing product, user, and transaction data.

Strengths:

- Scalable: Servers can handle millions of users.
- Maintainable: Layers isolate changes (e.g., database updates don't break UI).
- Clear separation of concerns.

Limitations:

- Latency: Requests pass through multiple layers, potentially slowing response.
- Complexity: More components to maintain.

Example 2:

Netflix Data Pipeline

- Pipe-and-Filter:
 - Data passes through stages: raw event ingestion → filtering → aggregation → recommendation scoring.
- Repository:
 - Processed data is stored in a central repository, allowing multiple services such as: recommendation engine, analytics dashboards to access the same data.

Strengths:

- Reusable components: Filters can be modified independently.
- Centralised storage: Repository allows multiple teams to access consistent data.
- Scalable: Pipeline can run in parallel, handling huge volumes of streaming data.

Limitations:

- Latency: Writing and reading from the repository can introduce delays.
- Complexity: Coordinating filters and ensuring data consistency can be tricky.